Name: _____
Surname: _____
Date: _____

Intructions

Always assume each code is run independently, so variables do not persist between exercises.

Questions 1-6: Multiple Choice (5 points each)

Circle the correct answer for each question, as each question has **only one** correct answer. A correct answer awards 5 points. There is no penalty for wrong choices.

Questions 7-9: Debugging (10 points each)

These exercises show code cells with one or more errors. You have to find these errors, explain why they happen, and suggest a fix to them.

Questions 10-11: Open Problems (20 points each)

In these exercises, you will encounter problems that are more open-ended and may not have a single correct answer. Your task is to explain how you would approach each problem by breaking it down into smaller, manageable steps.
Describe how you would structure your code, which logical steps are needed, and what kind of Python tools or constructs (e.g., loops, conditionals, functions, lists, etc.) you would use. You are not required to write full code, but doing so is encouraged if it helps clarify your ideas.

---

1. After we run the code below, what will be the value of "x"?

```
s = "1.5 to the power of 2 is 2.25"
x = 10 * float(s[-4:])
```

   (a) `x = -40.0`
   (b) `x = 15.0`
   (c) `x = 22.5`
   (d) The code will raise an error.

2. Which statement is **FALSE** regarding keyword arguments in Python?

   (a) Keyword arguments are defined by specifying the parameter name followed by an equal sign and the value.
   (b) Keyword arguments are always required.
   (c) You are not allowed to put keyword arguments before positional arguments.
   (d) Keyword arguments are used to set default parameter values.

3. After we run the code below, what will be the value of "x"?

```
s = "1.5 to the power of 2 is 2.25"
count = 0
for x in s:
  if x in "123456789":
    count += 1
```

   (a) 0
   (b) 5
   (c) 6
   (d) The code will raise an error.

4. Consider the following block of code. What will be the console's output?

```
def exercise(x: list) -> tuple:
  # Reverse the list
  rev = x[::-1]
  return tuple(rev)

result = exercise("hola")
```

   (a) Python will print "hol".
   (b) Python will print "aloh".
   (c) The code runs fine but the user won't see any output.
   (d) The code will raise an error.

5. Which one of these statements is **FALSE** about tuples?

   (a) You can create a tuple with mixed data types.
   (b) Tuples are ordered and can be indexed.
   (c) Tuples are mutable.
   (d) Tuples can contain duplicated values.

6. After we run the code below, what will be the value of "ls_new"?

```
ls = [1, 2, 3]
ls_new = []
while len(ls) > 0:
  x = ls[0]
  ls_new.extend([x, x])
  ls.remove(x)
```

   (a) `ls_new = []`
   (b) `ls_new = [1, 2, 3]`
   (c) `ls_new = [1, 1, 2, 2, 3, 3]`
   (d) The code will never end (infinite loop).

Answers: 1-c, 2-b, 3-b, 4-c, 5-c, 6-c

7. The function below is designed to detect palindromes. It uses an auxiliary function inside. When we run the code, it raises an error on line 10: "`NameError: name "lower_and_join" is not defined`". Identify and fix the issue(s).

```
1  def is_palindrome(input_str: str) -> bool:
2
3    def lower_and_join(input_str: str) -> str:
4      return input_str.lower().replace(" ", "")
5
6    input_str = lower_and_join(input_str)
7    return input_str == input_str[::-1]
8
9  sentence = "Was it a car or a cat I saw"
10 print(lower_and_join(sentence))  # Expected: wasitacaroracatisaw
11 print(is_palindrome(sentence))  # Expected: True
```

Answer:

The error comes from the scope of `lower_and_join`. This function is defined **inside** `is_palindrome`, so its name is local: it only exists while `is_palindrome` is running. When we call it from the global scope in line 10, Python cannot find that name and raises a `NameError`. *(+5 points)*

We fix it by moving the definition of `lower_and_join` out of `is_palindrome`, into the global scope (removing its indentation, and defining it before it is used). Then both the `print` in line 10 and the function `is_palindrome` can use it. Deleting line 10 is not a good fix, as the expected output includes its result. *(+5 points)*

(Optional, fixed code)

```
1  def lower_and_join(input_str: str) -> str:
2      return input_str.lower().replace(" ", "")
3
4  def is_palindrome(input_str: str) -> bool:
5      input_str = lower_and_join(input_str)
6      return input_str == input_str[::-1]
7
8  sentence = "Was it a car or a cat I saw"
9  print(lower_and_join(sentence))  # Expected: wasitacaroracatisaw
10 print(is_palindrome(sentence))  # Expected: True
```

8. The code below is intended to accumulate values in a list every time the function is called. However, it behaves unexpectedly. Identify and fix the issue(s).

```python
def accumulate(value: int, lst: list=[]) -> list:
  lst.append(value)
  return lst

print(accumulate(1))  # Output: [1], Expected: [1]
print(accumulate(2))  # Output: [1, 2], Expected: [2]
print(accumulate(3, lst=[4]))  # Output: [4, 3], Expected: [4, 3]
```

Answer:

The problem is the default value of the parameter, `lst: list=[]`. Python creates that default list **only once**, when the function is defined, and not every time it is called. Therefore, all the calls that do not provide `lst` use (and modify) the same list. This is why the second call returns `[1, 2]` instead of `[2]`. It happens because lists are mutable. *(+5 points)*

The standard fix is to use an immutable default value, `None`, and create a new list inside the function when needed: `if lst is None: lst = []`. This creates a fresh list in every call. The third call, which provides its own list, keeps working as before. *(+5 points)*

(Optional, fixed code)

```python
def accumulate(value: int, lst: list=None) -> list:
    if lst is None:
        lst = []
    lst.append(value)
    return lst
```

9. The following function is intended to find and return the maximum even number from a given list of integers. However, it sometimes returns incorrect results. Identify the issue(s) and propose a fix.

```python
def max_even_number(numbers: list[int]) -> int:
  max_even = None
  for num in numbers:
    if num % 2 == 0 and (max_even is None or num > max_even):
      max_even = num
    if max_even is None:
      return "No even number found"
    else:
      return max_even

nums = [1, 2, 3, 4]
print(max_even_number(nums))  # Prints "No even number found"

nums = [2, 4, 6]
print(max_even_number(nums))  # Prints 2
```

Answer:

The problem is the indentation of the last `if-else`. It is inside the for loop, so the function returns at the end of the *first* iteration, without looking at the rest of the numbers. For `[1, 2, 3, 4]`, the first number (1) is odd, so `max_even` is still `None` and the function returns the message right away. For `[2, 4, 6]`, it returns the first even number (2) instead of the maximum. *(+5 points)*

We fix it by moving the `if-else` out of the loop (reducing its indentation), so that it runs once, after all the numbers have been checked. *(+5 points)*

Comment: The type hint says that the function returns an `int`, but it returns a string when there are no even numbers. It would be better to return `None` in that case. This is not required here.

(Optional, fixed code)

```python
def max_even_number(numbers: list[int]) -> int:
    max_even = None
    for num in numbers:
        if num % 2 == 0 and (max_even is None or num > max_even):
            max_even = num
    if max_even is None:
        return "No even number found"
    else:
        return max_even
```

10. The mathematical constant e, approximately equal to 2.71828, is fundamental in many areas of mathematics. It can be defined in various ways, two of which are:

    (a) **Limit Definition**:

    $$e = \lim_{n \to N} \left(1 + \frac{1}{n}\right)^n$$

    (b) **Series Expansion** (Maclaurin Series for e^x at $x = 1$):

    $$e = \sum_{n=0}^{N} \frac{1}{n!} = 1 + 1 + \frac{1}{2!} + \frac{1}{3!} + \frac{1}{4!} + \dots$$

    Where $N \to \infty$. Your task is to implement two functions, one for each of the definitions above. Both functions will utilize the argument `n` as the upper limit for approximation (N in the equations above).

Answer:

We need two functions, `e_limit(n)` and `e_series(n)`. Both receive the integer `n` and return a float, the approximation of e. *(+2 points)*

Limit definition. This one is a direct translation of the formula, and it does not need a loop: `return (1 + 1/n) ** n`. We need parentheses around `1 + 1/n`, and `n` must be greater than 0. The larger `n` is, the better the approximation. *(+5 points)*

Series expansion. We need to add the terms $1/n!$ for every n from 0 up to N (included). We initialize `total = 0`, and loop with `for i in range(n + 1)`, where the "+1" is needed to include the last term. *(+4 points)*

We do not have a factorial function, so we compute it inside the same loop. We initialize `factorial = 1` before the loop. In each iteration, if `i` is greater than 0 we update `factorial = factorial * i`. This works because $i! = i \cdot (i-1)!$, and because $0! = 1$. *(+5 points)*

Then, in each iteration we add the term to the total with `total += 1 / factorial`. After the loop, we return `total`. *(+4 points)*

Comment: We could also compute each factorial with a second (nested) loop. It gives the same result, but it is slower.

(Optional, code)

```python
def e_limit(n: int) -> float:
    return (1 + 1/n) ** n


def e_series(n: int) -> float:
    total = 0
    factorial = 1
    for i in range(n + 1):
        # Update the factorial (0! = 1, so we skip i = 0)
        if i > 0:
            factorial = factorial * i
        total += 1 / factorial
    return total
```

11. You have a dictionary that lists people and their hobbies, as shown below:

```
dict_people = {
  "Amy": ["cinema", "dancing", "health", "history", "travel"],
  "Martha": ["health", "running", "travel"],
  "Rory": ["cinema", "dancing", "history", "running"]
}
```

Design a function named `match_people(dict_people)`. This function should take a dictionary (like the one above) as input and print all pairs of people who share at least one hobby. For example, given the dictionary above, the output should be:

```
"Amy and Martha: health, travel"
"Amy and Rory: cinema, dancing, history"
"Martha and Rory: running"
```

Answer:

The function receives the dictionary, and it **prints** the results (no `return` is required). *(+2 points)*

We need to compare every person against every other person, but each pair only once. First, we get the names as a list, `names = list(dict_people.keys())`. Then we use a nested loop: the outer loop is `for i in range(len(names))`, and the inner loop is `for j in range(i + 1, len(names))`. Starting the inner loop at `i + 1` makes sure that we never compare a person with themselves, and that we never repeat a pair (we get "Amy and Martha" but not "Martha and Amy"). *(+7 points)*

Inside the inner loop, we convert the hobby lists of both people into sets, and use the intersection to find the hobbies they share: *(+5 points)*

```
shared = set(dict_people[name_a]).intersection(set(dict_people[name_b]))
```

We then use an if-clause to check that the set is not empty, `len(shared) > 0`. If so, we print the message using an f-string. As the hobbies are in a set, we sort them first, and use the method `join` to separate them with commas: `", ".join(sorted(shared))`. *(+6 points)*

(Optional, code)

```
def match_people(dict_people: dict) -> None:
    names = list(dict_people.keys())
    # Compare every pair of people once
    for i in range(len(names)):
        for j in range(i + 1, len(names)):
            name_a = names[i]
            name_b = names[j]
            # Hobbies in common
            shared = set(dict_people[name_a]).intersection(set(dict_people[
    name_b]))
            # Print only if there is at least one
            if len(shared) > 0:
                print(f"{name_a} and {name_b}: {', '.join(sorted(shared))}"
    )
```

Cheatsheet

Given a number named `x`:

- `x % y` returns the remainder of the division $\frac{x}{y}$.
- `x // y` returns the quotient of the division $\frac{x}{y}$.
- `abs(x)` returns the absolute value of `x`.

Given a string named `s`:

- `int(s)` returns an integer. Fails if `s` has invalid characters.
- `float(s)` returns a floating number. Fails if `s` has invalid characters.
- `s.split(" ")` split a string into a list where each word is a list item.
- `s.count(<pattern>)` returns the number of non-overlapping occurrences of `<pattern>`.
- `s.find(<pattern>, <start>)` finds the first occurrence of the specified `<pattern>`, starting from index `<start>`. Returns `-1` if the pattern is not found.

Given a list or tuple named `ls`:

- `len(ls)` returns the number of elements in that list.
- `sum(ls)` returns the sum of the elements of that list.
- `sorted(ls)` returns a sorted version of that list.
- `tuple(ls)` / `set(ls)` returns a tuple / set using the elements of that list.
- `", ".join(ls)` returns a string with the elements of the list separated by commas.

Given a set named `a`:

- `a.add(x)` adds the element `x` to the set `a`.
- `a.union(b)` returns the union of set `a` with another set `b`.
- `a.intersection(b)` returns the intersection of set `a` with another set `b`.
- `a.difference(b)` returns the difference of set `a` with another set `b`.

Given a dictionary named `d`:

- `list(d.keys())` returns a list with the keys of `d`.
- `list(d.values())` returns a list with the values of `d`.
- `list(d.items())` returns a list with the tuples (key, value) of `d`.

Some useful build-in functions:

- `type(x)` returns the type of `x` (int, float, list...).
- `range(start, stop)` generates numbers from `start` until `stop`, not including `stop`.